# Applied Data Science with R Capstone project

< Jimin Kim >

<Feb 3, 2023>

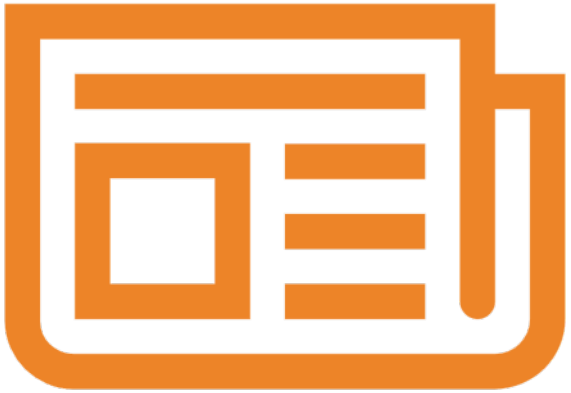
# Outline

- Introduction
- Methodology
- Results
- Conclusion
- Appendix

# Introduction

- Using Bike Sharing, Weather, and World City data, we build a model to predict the number of bike rentals at a certain city given a set of weather condition.
- Visualize the data analysis and create an interactive dashboard.

# Methodology

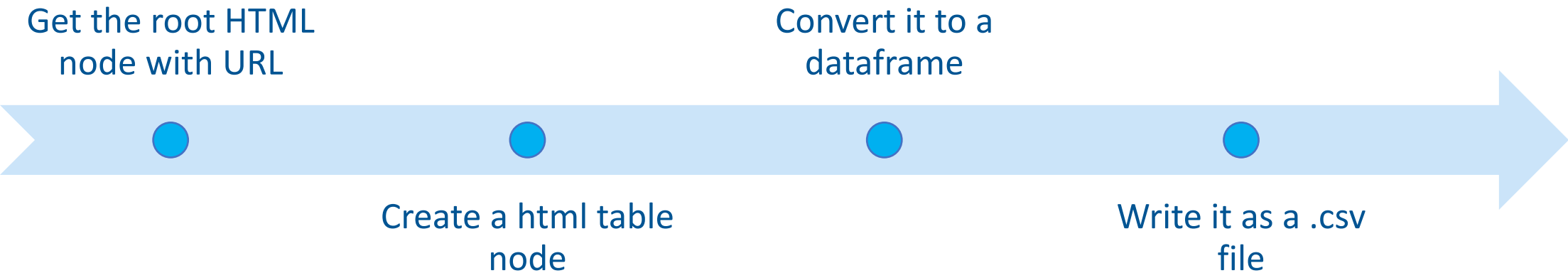


- Perform data collection
- Perform data wrangling
- Perform exploratory data analysis (EDA) using SQL and visualization
- Perform predictive analysis using regression models
- Build a R Shiny dashboard app

# Methodology

# Data collection

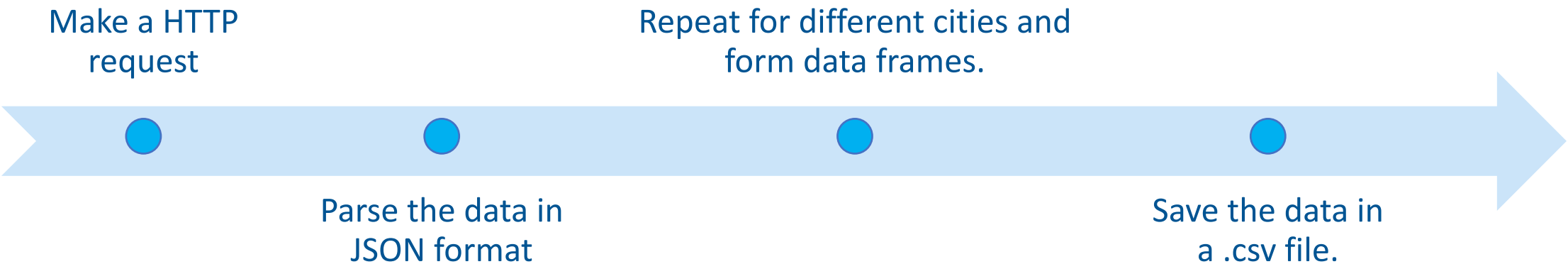
## Bike sharing systems in Seoul




Screenshots of Notebook code cell and cell output used for OpenWeatherAPI and Webscraping are added to the Appendix section for peer-review.

# Data collection

**Weather data by cities** *using OpenWeather API*




Screenshots of Notebook code cell and cell output used for OpenWeatherAPI and Webscraping are added to the Appendix section for peer-review.

# Data wrangling *with Regular Expression*

Goal: Standardizing columns & cleaning up unnecessary string values

Uppercase the column names → Remove reference links → Extract the numeric value

# Data wrangling *with dplyr*

Goal: Drop missing values, new variables for categorical values, & scaling data

Detect missing values and drop them → Create indicator variables → Normalize data

# EDA with SQL

- Using SQL, we print the selected part of the big data we collected in Data Collection.
- Find relative variable to the number of bike rentals.

# EDA with data visualization

- Create charts to visualize the relationships between the rented bike count and other variables.
- Describe the patterns of the charts.

# Predictive analysis

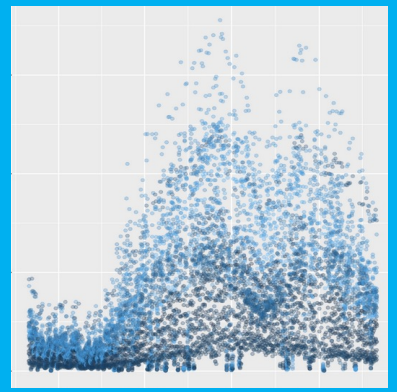
- Built a baseline regression model and improve it by adding polynomial terms, interaction terms, and regularization.

- Evaluate the model performance by comparing RMSE and R-squared reported values.

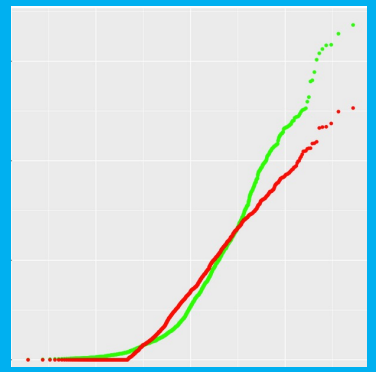
# Build a R Shiny dashboard

- Visualize weather forecast data and predicted hourly bike-sharing demand for the following cities: New York, USA, Paris, France, Suzhou, China, and London, UK.
- Use ggplot to render some more detailed plots such as bike-sharing prediction trend, temperature trend, humidity and bike-sharing demand prediction correlation, when users zoom-in to a city.

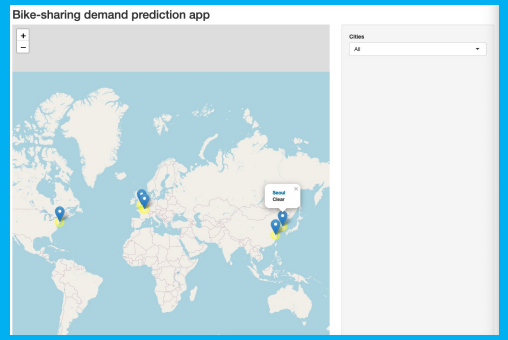
# Results



Exploratory data



Predictive analysis




A dashboard

# EDA with SQL

# Busiest bike rental times

Find dates and hours which had the most bike rentals

```
# provide your solution here
query = "SELECT DATE, HOUR, RENTED_BIKE_COUNT FROM SEOUL_BIKE_SHARING
            WHERE RENTED_BIKE_COUNT = (SELECT MAX(RENTED_BIKE_COUNT) FROM SEOUL_BIKE_SHARING)"
sqlQuery(conn,query)
```

A data.frame: 1 × 3

| | DATE | HOUR | RENTED_BIKE_COUNT |
|---|---|---|---|
| | <fct> | <int> | <int> |
| 1 | 19/06/2018 | 18 | 3556 |

The most bike rentals were 3556 at 19th of June in 2018.

# Hourly popularity and temperature by seasons

Find hourly popularity and temperature by season

```
# provide your solution here
query = "SELECT SEASONS, AVG(TEMPERATURE) as avg_temp, AVG(RENTED_BIKE_COUNT) as avg_bike_rentals FROM SE
            GROUP BY SEASONS"
sqlQuery(conn,query)
```

A data.frame: 4 × 3

| | SEASONS | AVG_TEMP | AVG_BIKE_RENTALS |
|---|---|---|---|
| | <fct> | <dbl> | <int> |
| 1 | Autumn | 13.821167 | 924 |
| 2 | Spring | 13.021389 | 746 |
| 3 | Summer | 26.587274 | 1034 |
| 4 | Winter | -2.540463 | 225 |

Summer is the most popular time for bike rentals and the average temperature of the season is 26.

# Rental Seasonality

## Rental Seasonality

```
# provide your solution here
query = "SELECT SEASONS, AVG(RENTED_BIKE_COUNT) as avg_bike_rentals, MIN(RENTED_BIKE_COUNT) AS MINIMUM,
        MAX(RENTED_BIKE_COUNT) AS MAXIMUM, STDDEV(RENTED_BIKE_COUNT) AS STANDARD_DEVIATION FROM SEOUL_BIK
            GROUP BY SEASONS"
sqlQuery(conn,query)
```

A data.frame: 4 × 5

| | SEASONS | AVG_BIKE_RENTALS | MINIMUM | MAXIMUM | STANDARD_DEVIATION |
|---|---|---|---|---|---|
| | <fct> | <int> | <int> | <int> | <dbl> |
| 1 | Autumn | 924 | 2 | 3298 | 617.3885 |
| 2 | Spring | 746 | 2 | 3251 | 618.5247 |
| 3 | Summer | 1034 | 9 | 3556 | 690.0884 |
| 4 | Winter | 225 | 3 | 937 | 150.3374 |

In Winter, the popularity of the bike rentals deeps down.

# Weather Seasonality

## Weather Seasonality

```
query <- "SELECT SEASONS, AVG(RENTED_BIKE_COUNT) AS BIKE_COUNT, AVG(TEMPERATURE) AS TEMPTERATURE,
AVG(HUMIDITY) AS HUMIDITY, AVG(WIND_SPEED) AS WIND_SPEED,
            AVG(VISIBILITY) AS VISIBILITY, AVG(DEW_POINT_TEMPERATURE) AS DEW_POINT_TEMP,
            AVG(SOLAR_RADIATION) AS SOLAR_RADIATION, AVG(RAINFALL) AS RAINFALL,
    AVG(SNOWFALL) AS SNOWFALL, RANK() OVER
    (ORDER BY AVG(RENTED_BIKE_COUNT)) AS RANK

    FROM SEOUL_BIKE_SHARING
            GROUP BY SEASONS
            "
sqlQuery(conn,query)
# provide your solution here
```

A data.frame: 4 × 11

| | SEASONS | BIKE_COUNT | TEMPTERATURE | HUMIDITY | WIND_SPEED | VISIBILITY | DEW_POINT_TEMP | SOLAR_RADIAT |
|---|---|---|---|---|---|---|---|---|
| | <fct> | <int> | <dbl> | <int> | <dbl> | <int> | <dbl> | <d |
| 1 | Winter | 225 | -2.540463 | 49 | 1.922685 | 1445 | -12.416667 | 0.2981 |
| 2 | Spring | 746 | 13.021389 | 58 | 1.857778 | 1240 | 4.091389 | 0.6803 |
| 3 | Autumn | 924 | 13.821167 | 59 | 1.492101 | 1558 | 5.150594 | 0.5227 |
| 4 | Summer | 1034 | 26.587274 | 64 | 1.609420 | 1501 | 18.750136 | 0.7612 |

# Bike-sharing info in Seoul

- The total Bike count in Seoul = 20000.
- The city info of the Seoul in WORLD_CITIES data is the following:

```
# provide your solution here
# total # of bikes available in Seoul....

query <- "SELECT BIKE_SHARING_SYSTEMS.BICYCLES, WORLD_CITIES.CITY,
        WORLD_CITIES.COUNTRY, WORLD_CITIES.LAT,
        WORLD_CITIES.LNG, WORLD_CITIES.POPULATION
        FROM WORLD_CITIES
        INNER JOIN BIKE_SHARING_SYSTEMS
        ON (WORLD_CITIES.CITY_ASCII = BIKE_SHARING_SYSTEMS.CITY) AND
                (WORLD_CITIES.CITY = 'Seoul')
        "
sqlQuery(conn,query)
```

A data.frame: 1 × 6

| | BICYCLES | CITY | COUNTRY | LAT | LNG | POPULATION |
|---|---|---|---|---|---|---|
| | <int> | <fct> | <fct> | <dbl> | <dbl> | <int> |
| 1 | 20000 | Seoul | Korea, South | 37.58 | 127 | 21794000 |

# Cities similar to Seoul

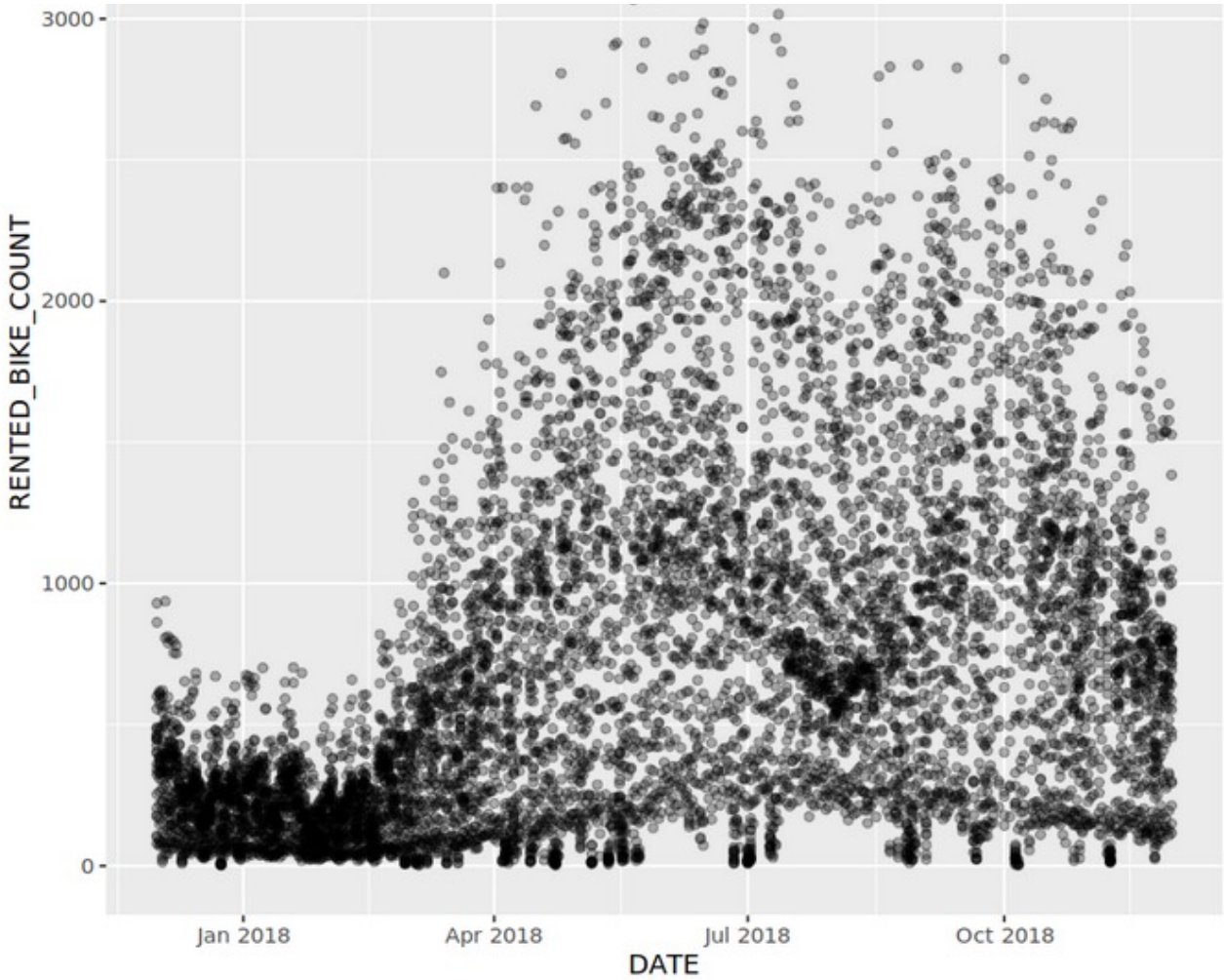
FIND ALL CITY NAMES AND COORDINATES WITH COMPARABLE BIKE SCALE TO SEOUL'S BIKE SHARING SYSTEM

PRESENT YOUR QUERY RESULT WITH A SHORT EXPLANATION HERE

# EDA with Visualization

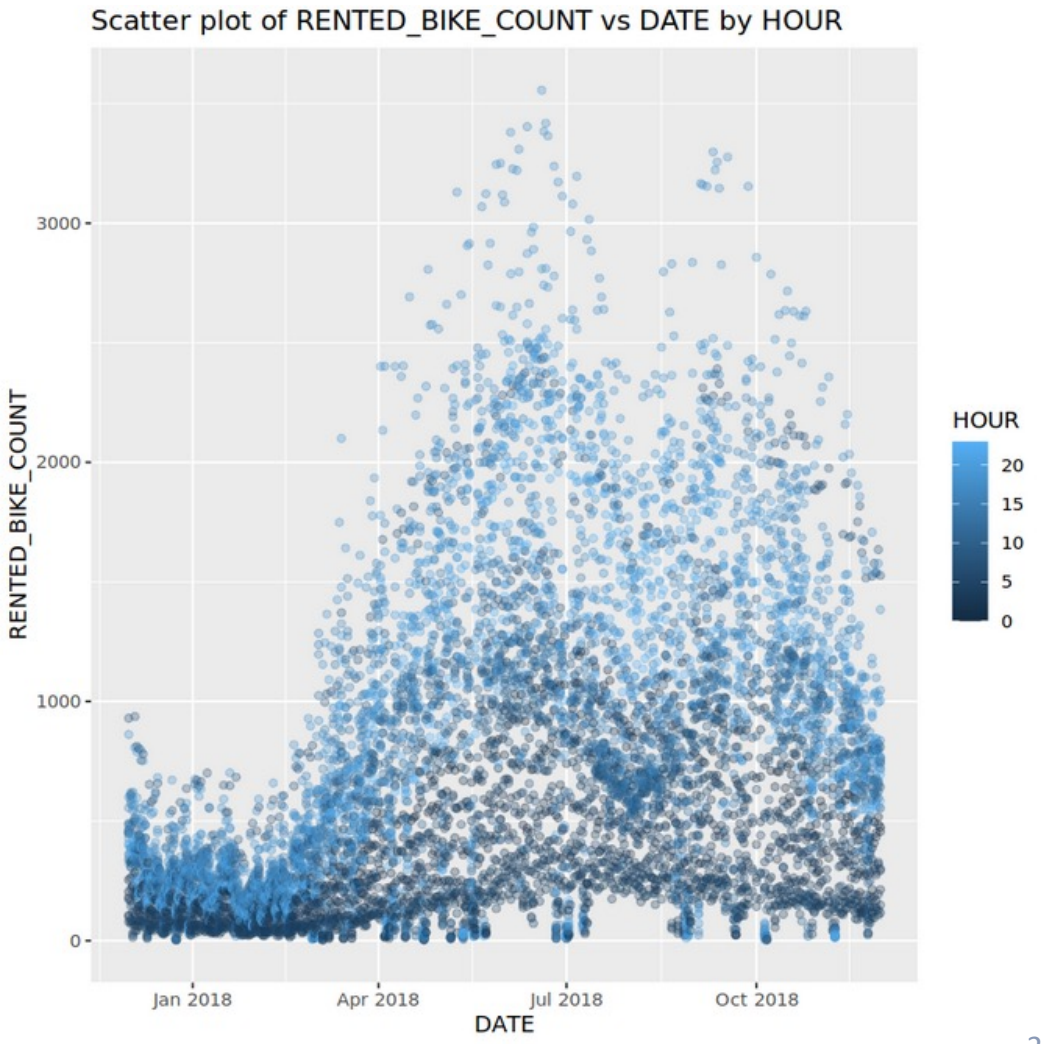
# Bike rental vs. Date

- The scatter plot on the right shows the popularity of the bike rentals.
- The number of rentals decreases in winter and increases in the warmer seasons.

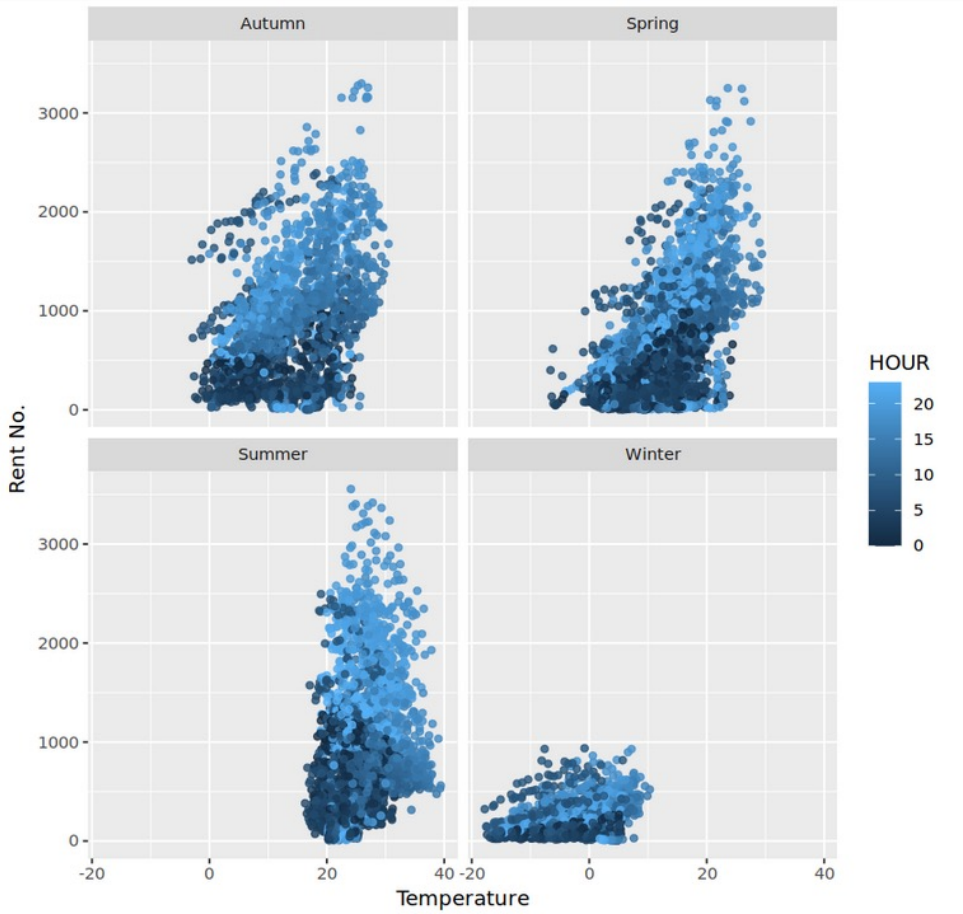


# Bike rental vs. Datetime

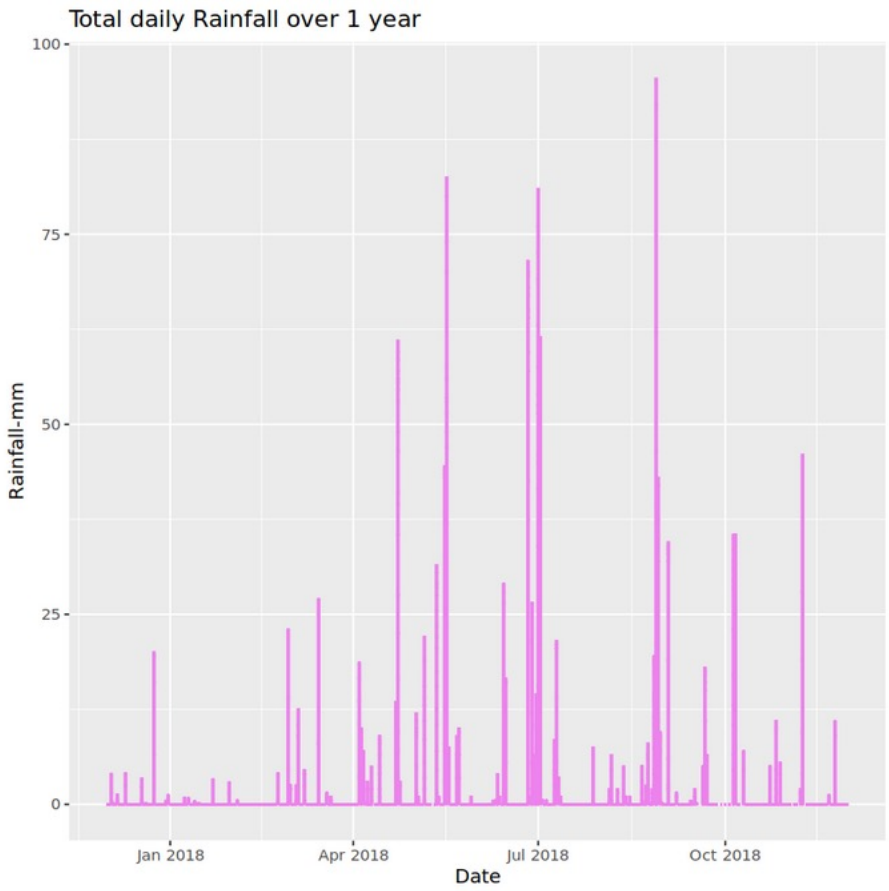
- The datatime of each data point is colored. The darker color means it's morning and the lighter color means it's afternoon.
- As expected, people uses the bike rental services during the day.

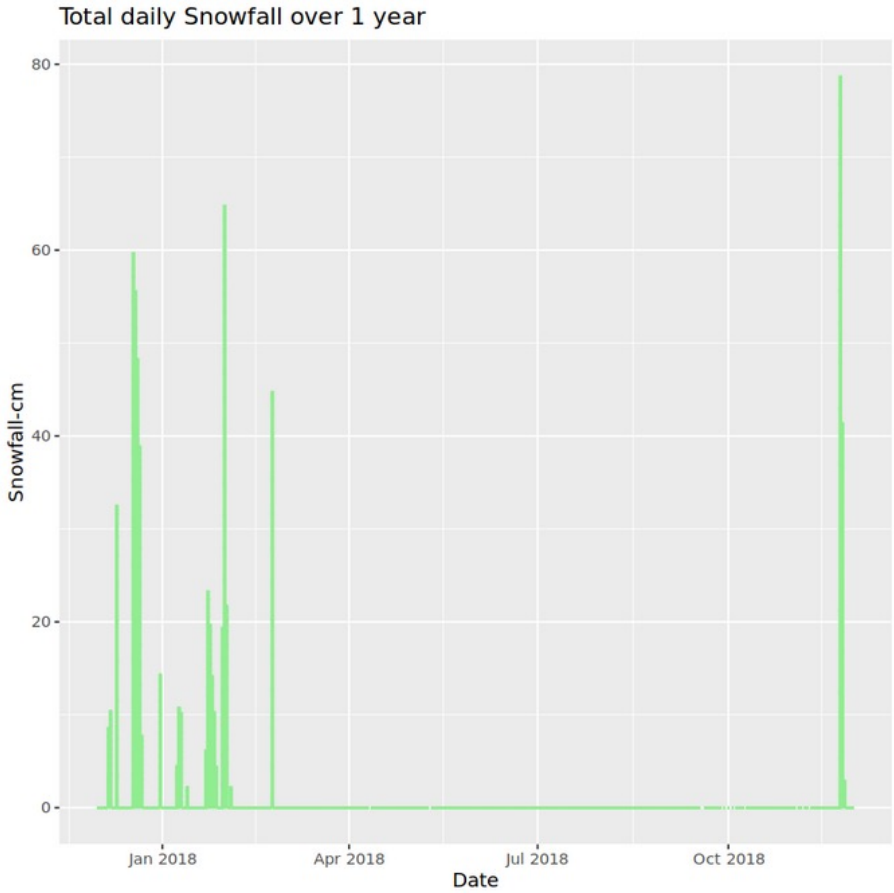


# The correlation between bike rentals and temperature by seasons



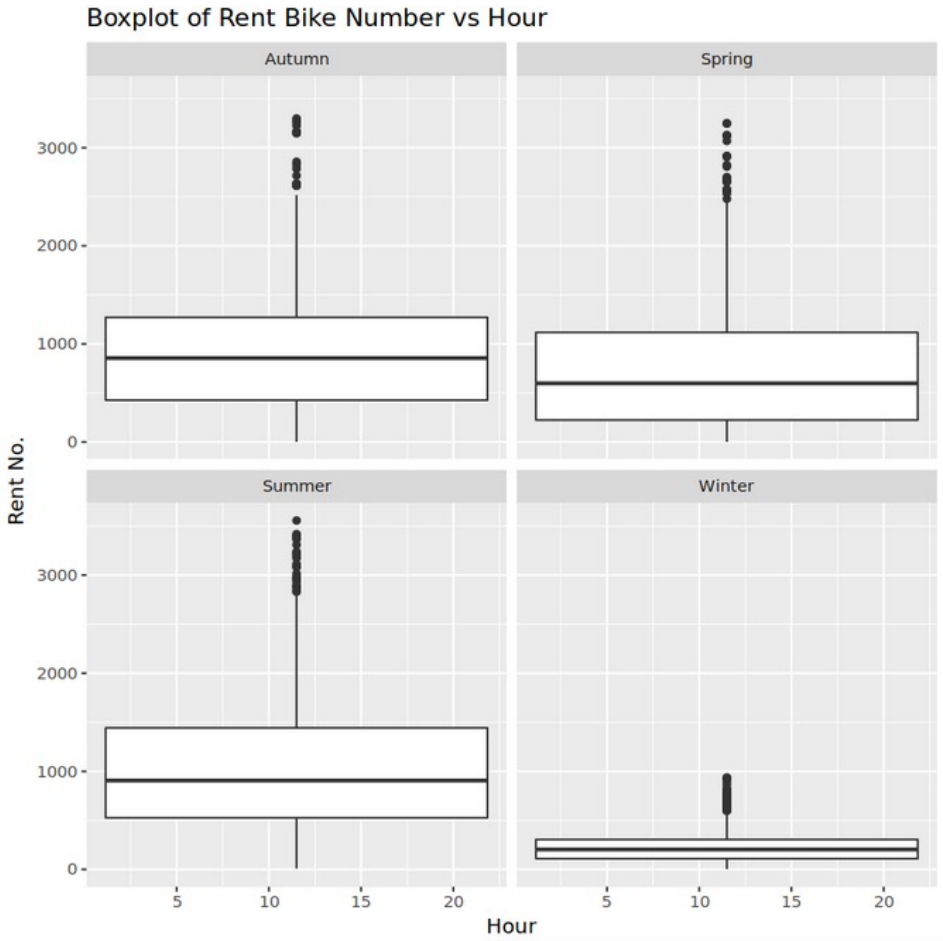
# Daily total rainfall and snowfall




| SEASONS | total_rainfall | total_snowfall |
|---|---|---|
| <chr> | <dbl> | <dbl> |
| Autumn | 227.9 | 123.0 |
| Spring | 403.8 | 0.0 |
| Summer | 559.7 | 0.0 |
| Winter | 70.9 | 534.6 |

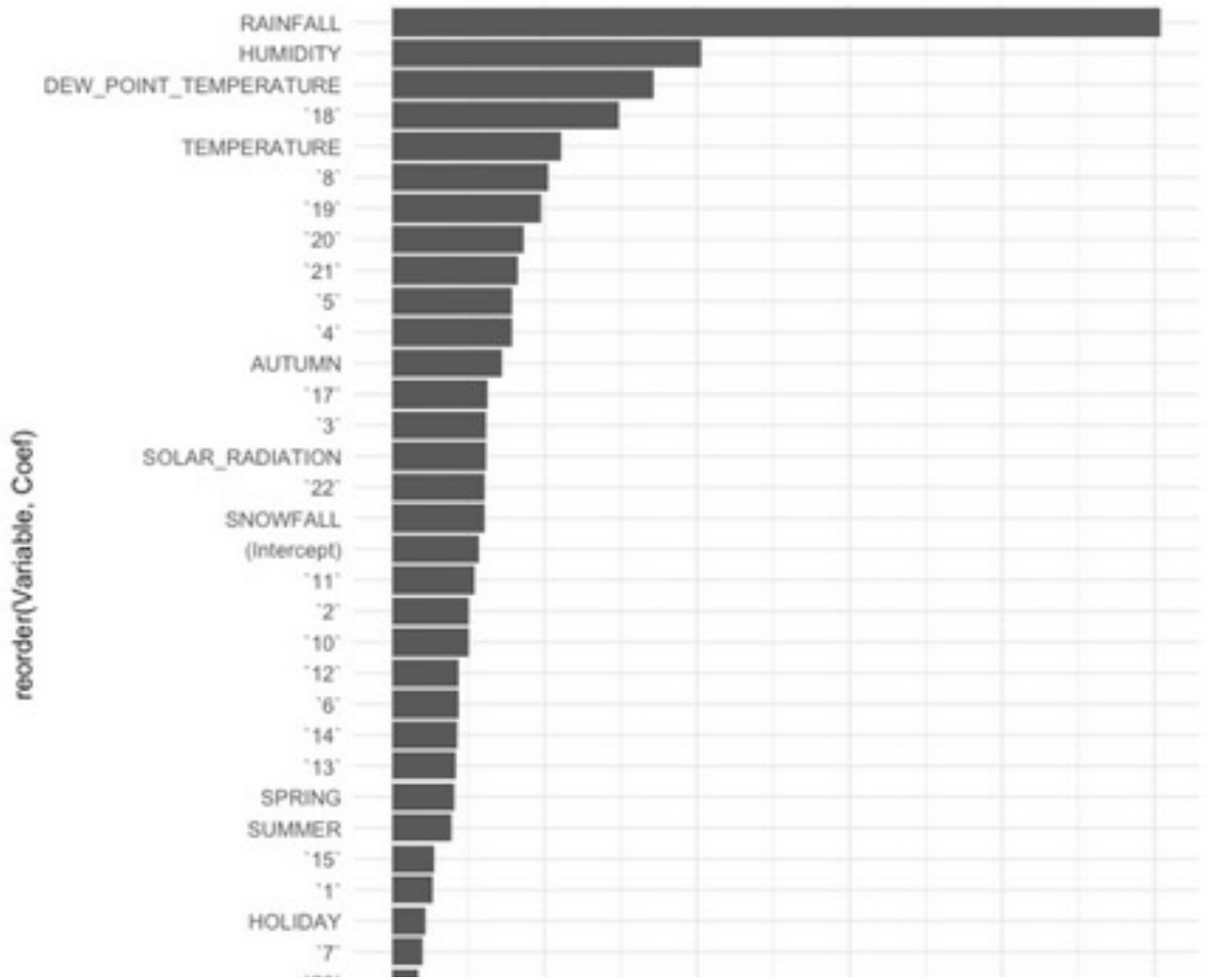
# Boxplots of bike rentals vs. hour grouped by seasons



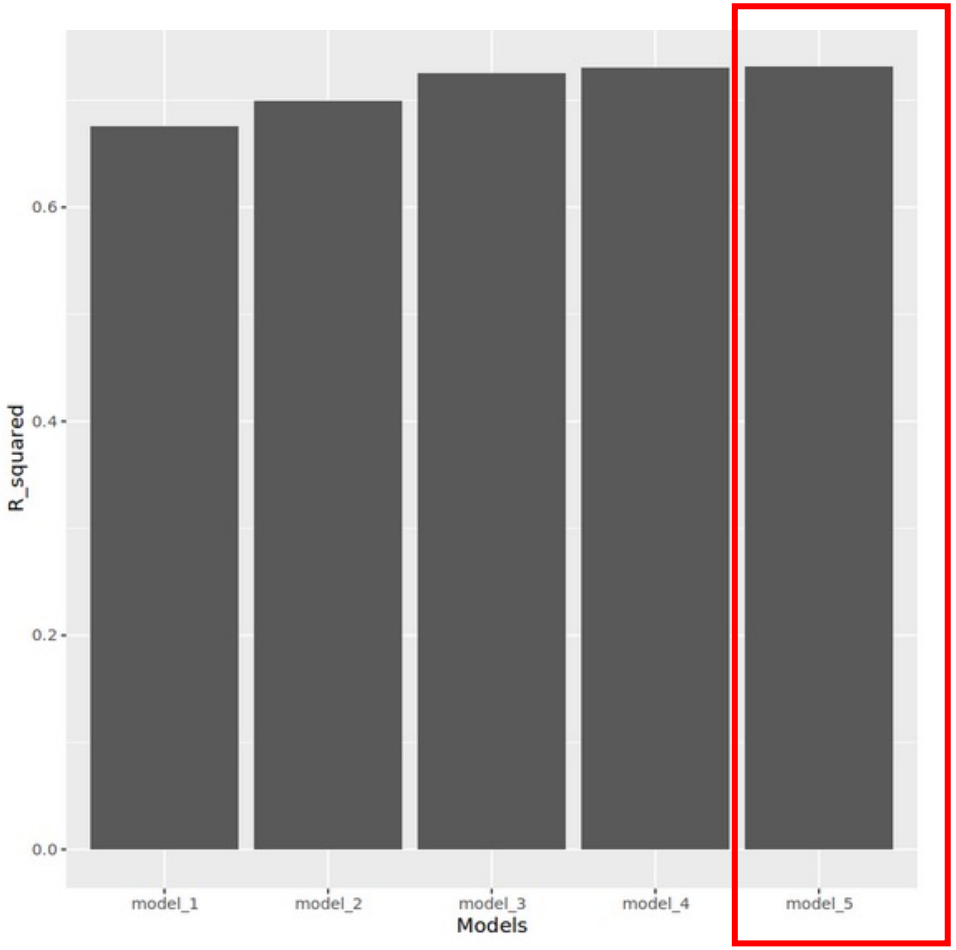
# Predictive analysis

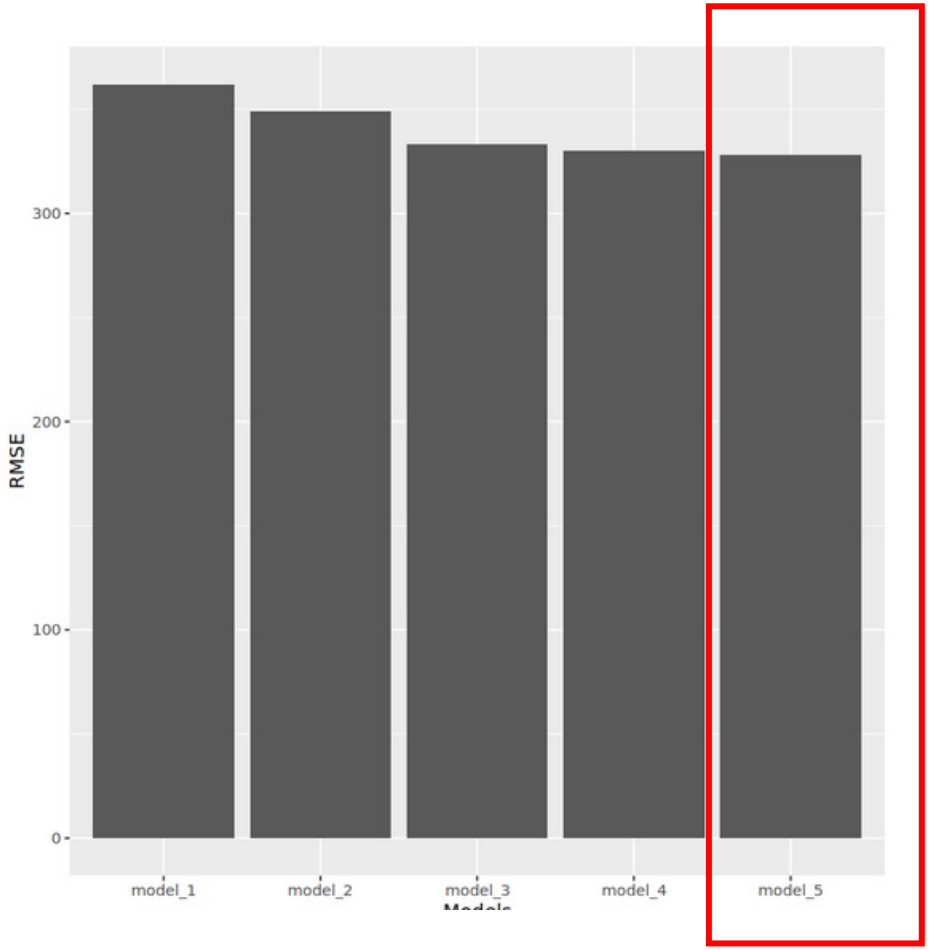
# Ranked coefficients

The most powerful factor to predict the number of bike rentals is if it’s rainy or not.



# Model evaluation




Model 5 performs the best.

# Best performing model

Best Model: RENTED_BIKE_COUNT ~.
+ SUMMER*SOLAR_RADIATION + poly(WIND_SPEED,4) + poly(VISIBILITY,2)
+ poly(DEW_POINT_TEMPERATURE,6)+ WINTER*SNOWFALL
+ poly(TEMPERATURE, 4) + HUMIDITY*RAINFALL

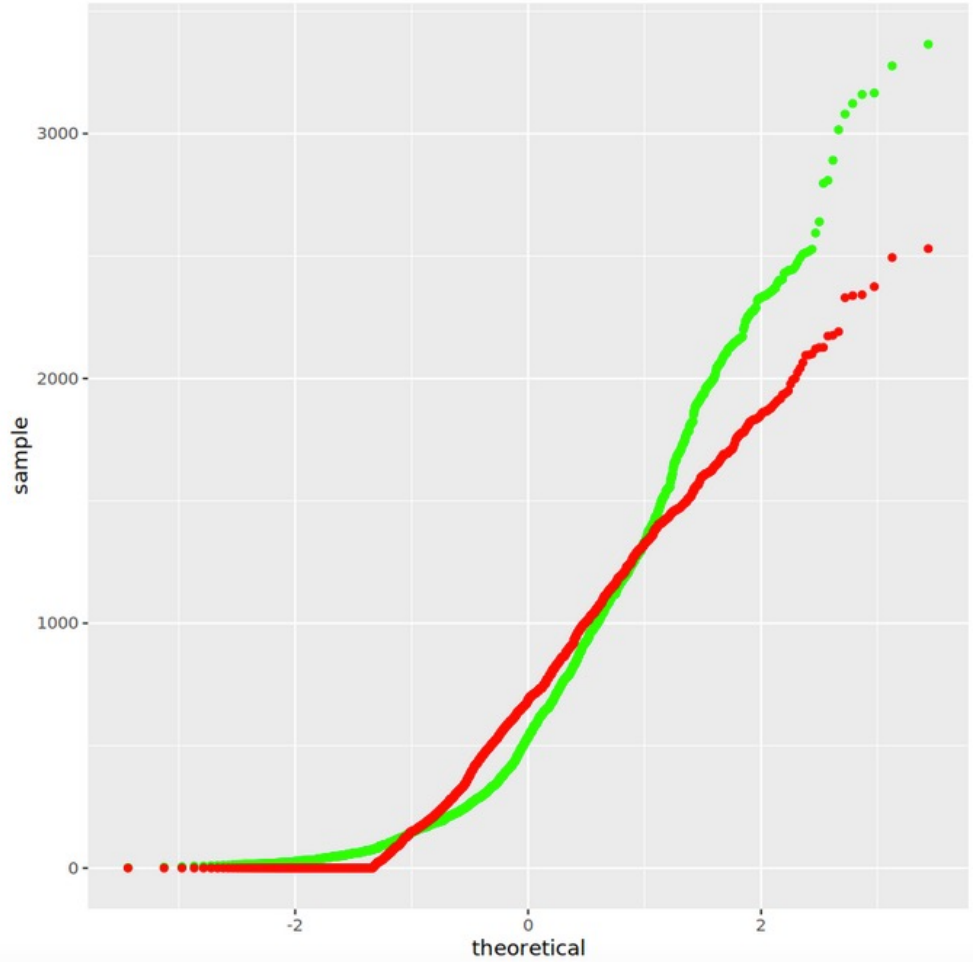
- RMSE is 328. ( < 330)
- R-squared is 0.732 ( > 0.72)

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rsq     standard       0.732
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard        328.
```

# Q-Q plot of the best model

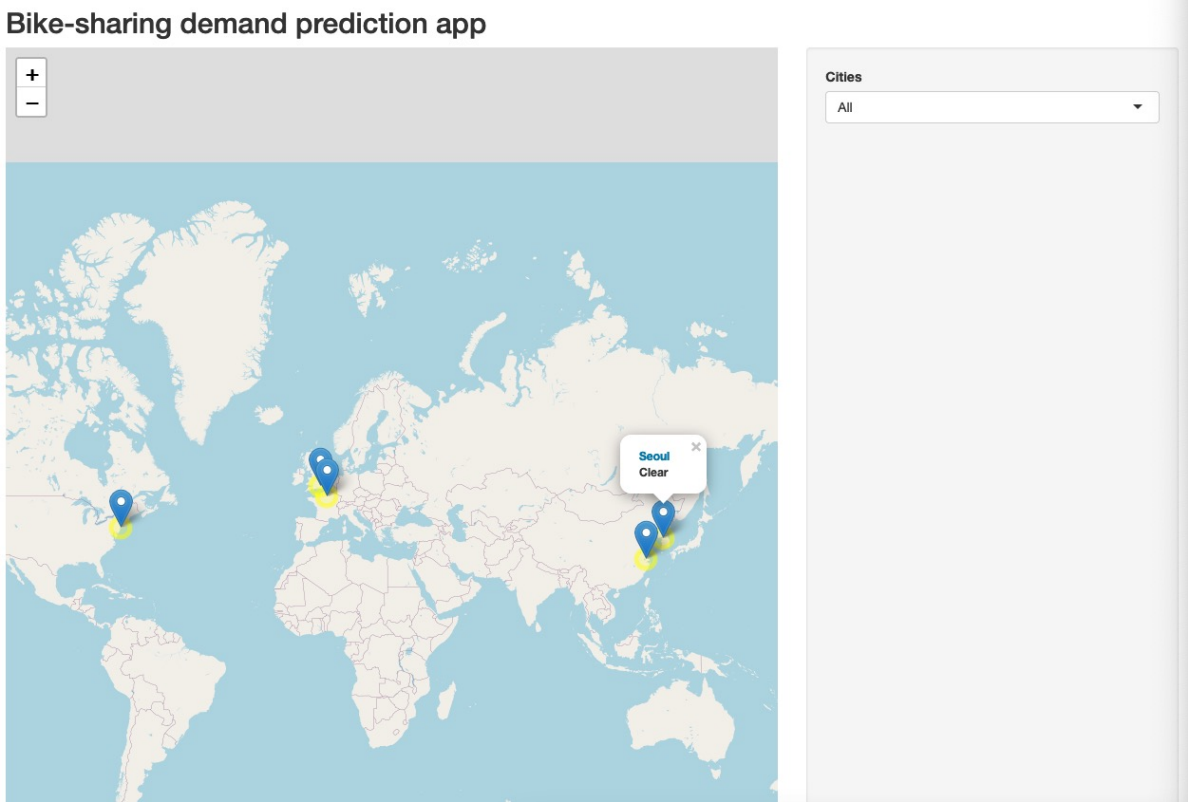
- The Q-Q plot of the best model's test results(red) vs the truths(green) .

```
[44]: ggplot(test_results) + stat_qq(aes(sample=truth), color='green') + stat_qq(aes(sample=.pred), color='red')
```



# Dashboard

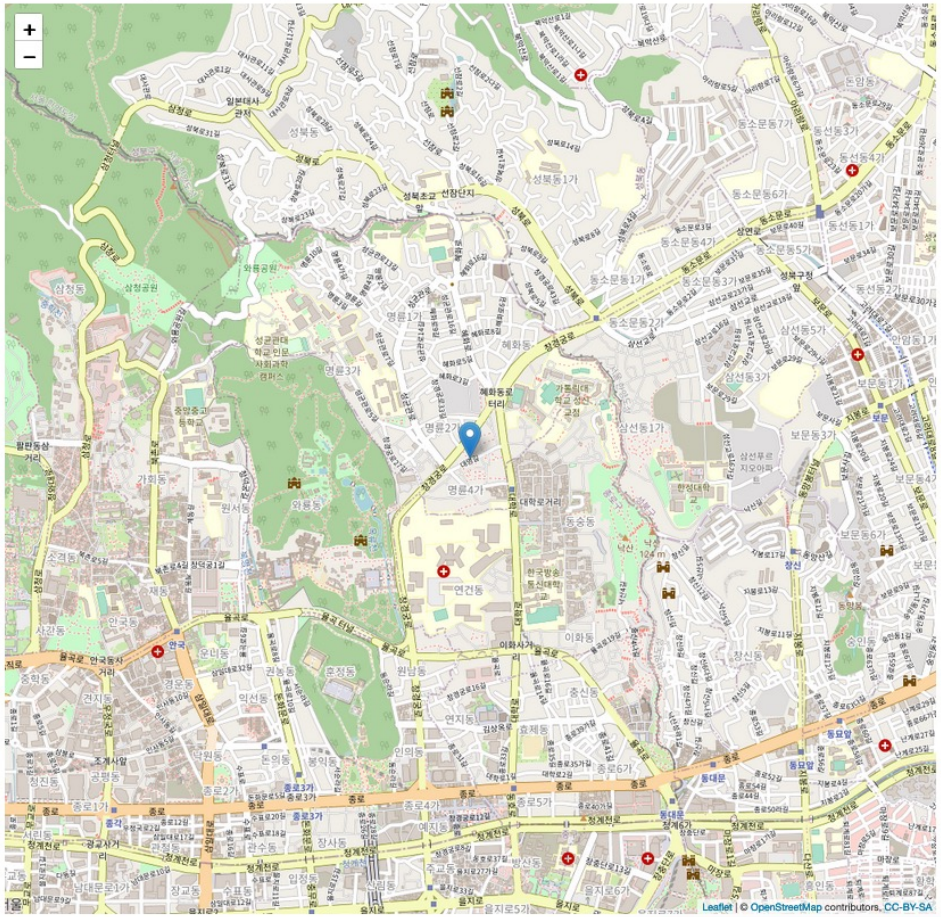
# Bike-sharing demand app

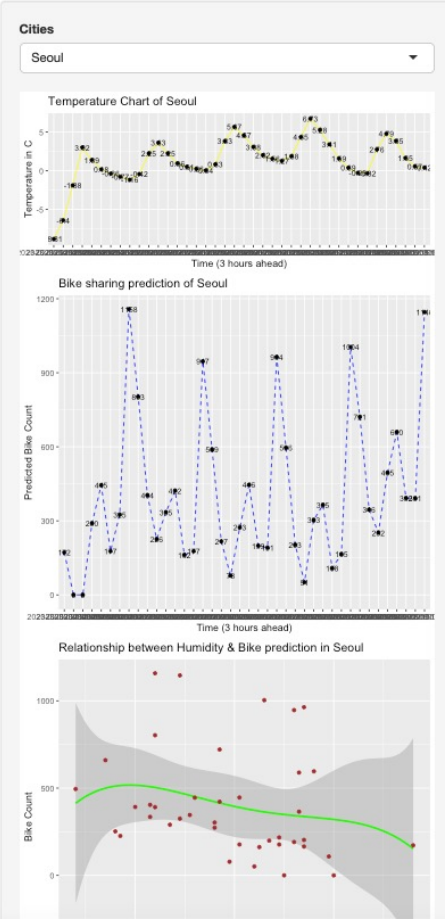



- Five cities are selected: Seoul, South Korea, New York, USA, Paris, France, Suzhou, China, and London, UK.
- Circle markers depend on the size of the predicted bike sharing.
- When clicking a location of the cities, the weather information is shown.

# When selected city is Seoul

Bike-sharing demand prediction app

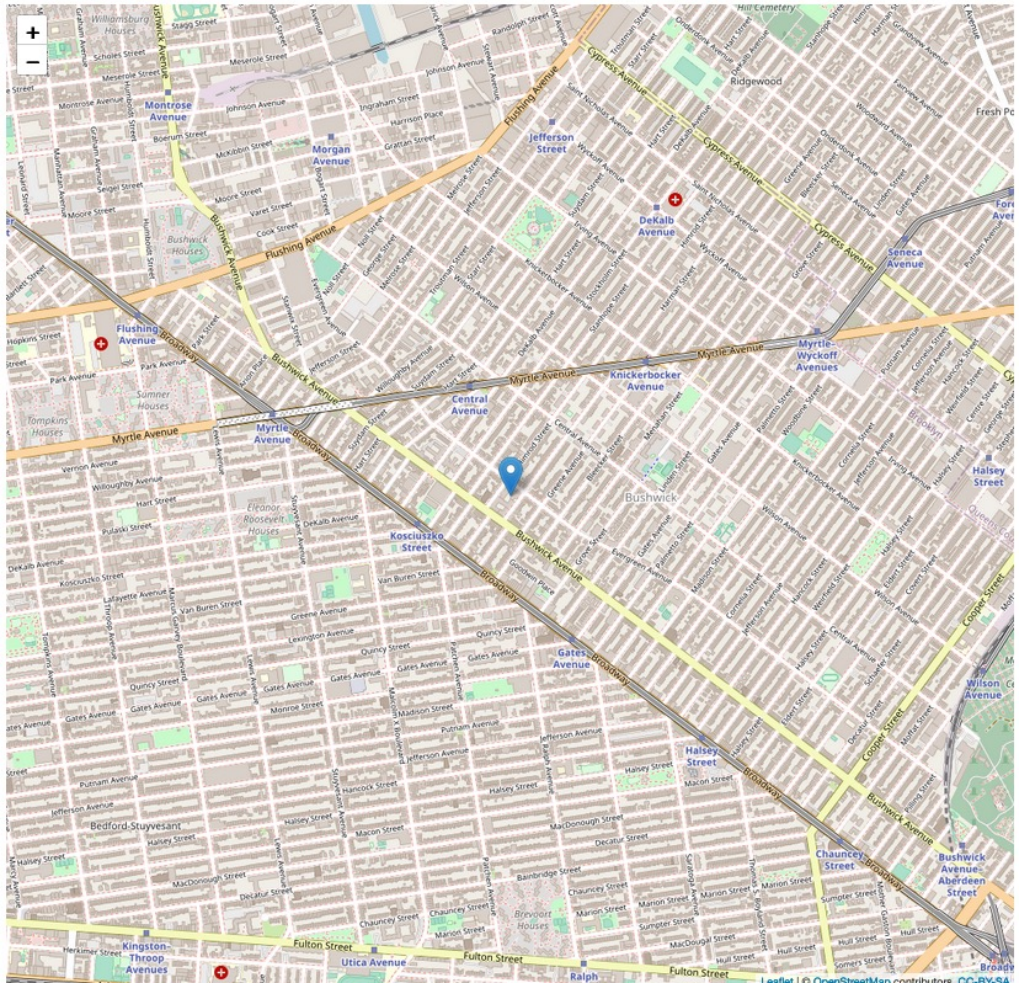


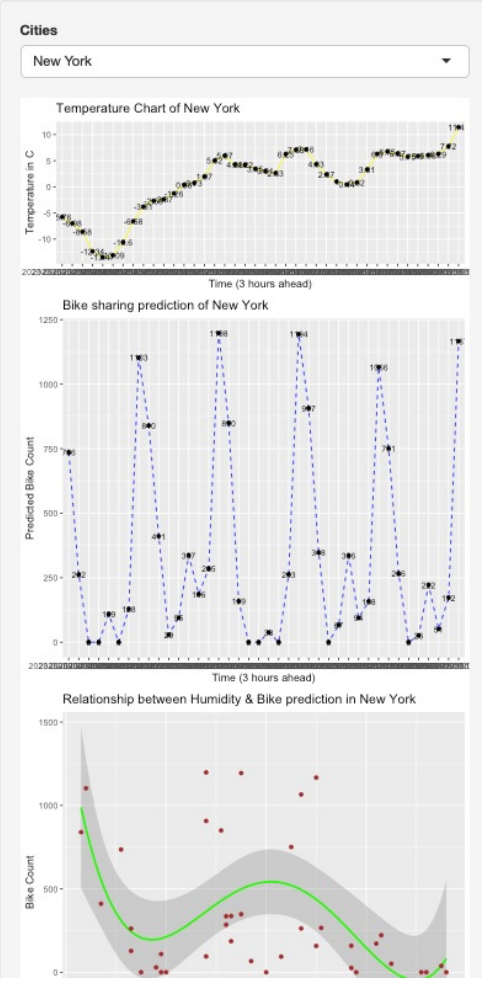




- When a city is selected (Seoul on the left), a zoomed-in map is shown.
- Moreover, three charts appear on the sidebar panel.

# Another example in NYC

Bike-sharing demand prediction app



# CONCLUSION

- The number of bike rentals in each city can be predicted based on the weather condition.

# APPENDIX

# Web scraping: Bike Sharing in Seoul

[21]:
```
url <- "https://en.wikipedia.org/wiki/List_of_bicycle-sharing_systems"
# Get the root HTML node by calling the `read_html()` method with URL
root_node <- read_html(url)
```

Note that this HTML page at least contains three child `<table>` nodes under the root HTML node. So, you will need to use `html_nodes(root_node, "table")` function to get all its child `<table>` nodes:

```
<html>
  <table>(table1)</table>
  <table>(table2)</table>
  <table>(table3)</table>
  ...
</html>
```

[ ]:
```
table_nodes <- html_nodes(root_node, "table")
```

You can use a `for` loop to print each table, and then you will see that the actual the bike sharing table is the second element `table_nodes[[2]]`.

[26]:
```
df<-html_table(table_nodes[[1]], fill = TRUE)
```

[28]:
```
head(df)
```

A data.frame: 6 × 10

| | Country | City | Name | System | Operator | Launched | Discontinued | Stations | Bicycles | Daily ridership |
|---|---|---|---|---|---|---|---|---|---|---|
| | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> |
| 1 | Albania | Tirana[5] | Ecovolis | | | March 2011 | | 8 | 200 | |
| 2 | Argentina | Buenos Aires[6][7] | Ecobici | Serttel Brasil[8] | Bike In Baires Consortium[9] | 2010 | | 400 | 4000 | 21917 |
| 3 | Argentina | Mendoza[10] | Metrobici | | | 2014 | | 2 | 40 | |
| 4 | Argentina | Rosario | Mi Bici Tu Bici[11] | | | 2 December 2015 | | 47 | 480 | |
| 5 | Argentina | San Lorenzo, Santa Fe | Biciudad | Biciudad | | 27 November 2016 | | 8 | 80 | |
| 6 | Australia | Melbourne[12] | Melbourne Bike | PBSC & 8D | Motivate | June 2010 | 30 November | 53 | 676 | |

# Web scraping: OpenWeahter API

## Coding Practice: Get the current weather data for a city using OpenWeather API

First import `httr` library

[16]:
```
# Check if need to install rvest` library
require("httr")

library(httr)
```

The API base URL to get current weather is https://api.openweathermap.org/data/2.5/weather

[31]:
```
# URL for Current Weather API
current_weather_url <- 'https://api.openweathermap.org/data/2.5/weather'
```

Next, let's create a list to hold URL parameters for current weather API

[32]:
```
# need to be replaced by your real API key
your_api_key <- "1bdbf5520b3379cfeb8e37eaffd3ad6c"
# Input `q` is the city name
# Input `appid` is your API KEY,
# Input `units` are preferred units such as Metric or Imperial
current_query <- list(q = "Seoul", appid = your_api_key, units="metric")
```

Now we can make a HTTP request to the current weather API

[33]:
```
response <- GET(current_weather_url, query=current_query)
```

If we check the response type, we can see it is in JSON format

[34]:
```
http_type(response)
```

'application/json'

JSON is an open standard file and data interchange format that uses human-readable text to store and transmit data objects. To HTTP response, you can use the `content()` function to parse it as a named list in R.

[35]:
```
json_result <- content(response, as="parsed")
```

Now let's print the JSON result.

[37]:
```
json_result
```

| | | |
|---|---|---|
| **$coord** | **$lon** | 126.9778 |
| | **$lat** | 37.5683 |
| **$weather** | 1. **$id** | 741 |
| | **$main** | 'Fog' |
| | **$description** | 'fog' |
| | **$icon** | '50n' |
| **$base** | 'stations' | |
| **$main** | **$temp** | 2.71 |
| | **$feels_like** | 1.28 |
| | **$temp_min** | -0.31 |
| | **$temp_max** | 3.78 |
| | **$pressure** | 1015 |
| | **$humidity** | 87 |
| **$visibility** | 200 | |
| **$wind** | **$speed** | 1.54 |
| | **$deg** | 310 |
| **$clouds** | **$all** = 75 | |
| **$dt** | 1675190268 | |
| **$sys** | **$type** | 1 |
| | **$id** | 8105 |
| | **$country** | 'KR' |
| | **$sunrise** | 1675204583 |
| | **$sunset** | 1675241677 |
| **$timezone** | 32400 | |
| **$id** | 1835848 | |
| **$name** | 'Seoul' | |
| **$cod** | 200 | |

# EDA with SQL

```r
for (dataset_name in dataset_list){
    # Read dataset
    dataset <- read_csv(dataset_name)
    # Standardized its columns:

    # Convert all column names to uppercase
    names(dataset) <- toupper(names(dataset))
    # Replace any white space separators by underscores, using the str_replace_all function
    names(dataset) <- str_replace_all(names(dataset), ' ', '_')
    # Save the dataset
    write.csv(dataset, dataset_name, row.names=FALSE)
}
```

```
Parsed with column specification:
cols(
  COUNTRY = col_character(),
  CITY = col_character(),
  NAME = col_character(),
  SYSTEM = col_character(),
  OPERATOR = col_character(),
  LAUNCHED = col_character(),
  DISCONTINUED = col_character(),
  STATIONS = col_character(),
  BICYCLES = col_character(),
  DAILY_RIDERSHIP = col_character()
```

```r
# remove reference link
remove_ref <- function(strings) {
    ref_pattern <- "\\[[A-z0-9]+\\]"
    # Replace all matched substrings with a white space using str_replace_all()
    result <- str_replace_all(strings, ref_pattern, "")
    # Trim the reslt if you want
    return(result)
}
```

*TODO:* Use the `dplyr::mutate()` function to apply the `remove_ref` function to the `CITY` and `SYSTEM` columns

```r
result <- sub_bike_sharing_df %>% mutate(CITY=remove_ref(CITY), SYSTEM = remove_ref(SYSTEM))
```

```r
bike_sharing_df <- bike_sharing_df %>% mutate(CITY=remove_ref(CITY), SYSTEM = remove_ref(SYSTEM))
```

*TODO:* Use the following code to check whether all reference links are removed:

```r
result %>%
    select(CITY, SYSTEM, BICYCLES) %>%
    filter(find_reference_pattern(CITY) | find_reference_pattern(SYSTEM) | find_reference_pattern(
```

A spec_tbl_df: 25 × 3

| CITY | SYSTEM | BICYC |
|---|---|---|
| <chr> | <chr> | <( |
| Brussels | 3 Gen. Cyclocity | 4115 |
| Limassol (& Agios Dometios, Aglandjia, Dali, Engomi, Latsia, Pallouriotissa, Strovolos) | 3 Gen. Smoove | 310[59] |
| Prague | NA | 500 |
| Prague 7 | 4 Gen. Ofo | |

```r
# Extract the first number
extract_num <- function(columns){
    # Define a digital pattern
    digitals_pattern <- "[0-9]+"
    # Find the first match using str_extract
    result <- str_extract(columns, digitals_pattern)
    # Convert the result to numeric using the as.numeric() function
    result <- as.numeric(result)
}
```

*TODO:* Use the `dplyr::mutate()` function to apply `extract_num` o

```r
# Use the mutate() function on the BICYCLES column
result <- sub_bike_sharing_df %>% mutate(BICYCLES=extract_num(BICYCLE
```

*TODO:* Use the summary function to check the descriptive statistics of

```r
summary(result$BICYCLES)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's 
      5     100     343    2012    1400   78000      76 
```

```
# provide your solution here
query = "SELECT DATE, HOUR, RENTED_BIKE_COUNT FROM SEOUL_BIKE_SHARING
            WHERE RENTED_BIKE_COUNT = (SELECT MAX(RENTED_BIKE_COUNT) FROM SEOUL_BIKE_SHARING)"
sqlQuery(conn,query)
```

A data.frame: 1 × 3

| | DATE | HOUR | RENTED_BIKE_COUNT |
|---|---|---|---|
| | <fct> | <int> | <int> |
| 1 | 19/06/2018 | 18 | 3556 |

```
# provide your solution here
query = "SELECT SEASONS, AVG(TEMPERATURE) as avg_temp, AVG(RENTED_BIKE_COUNT) as avg_bike_rentals FROM SE
            GROUP BY SEASONS"
sqlQuery(conn,query)
```

A data.frame: 4 × 3

| | SEASONS | AVG_TEMP | AVG_BIKE_RENTALS |
|---|---|---|---|
| | <fct> | <dbl> | <int> |
| 1 | Autumn | 13.821167 | 924 |
| 2 | Spring | 13.021389 | 746 |
| 3 | Summer | 26.587274 | 1034 |
| 4 | Winter | -2.540463 | 225 |

```
query <- "SELECT SEASONS, AVG(RENTED_BIKE_COUNT) AS BIKE_COUNT, AVG(TEMPERATURE) AS TEMPTERATURE,
AVG(HUMIDITY) AS HUMIDITY, AVG(WIND_SPEED) AS WIND_SPEED,
            AVG(VISIBILITY) AS VISIBILITY, AVG(DEW_POINT_TEMPERATURE) AS DEW_POINT_TEMP,
            AVG(SOLAR_RADIATION) AS SOLAR_RADIATION, AVG(RAINFALL) AS RAINFALL,
    AVG(SNOWFALL) AS SNOWFALL, RANK() OVER
    (ORDER BY AVG(RENTED_BIKE_COUNT)) AS RANK

    FROM SEOUL_BIKE_SHARING
            GROUP BY SEASONS
            "
sqlQuery(conn,query)
# provide your solution here
```

A data.frame: 4 × 11

| | SEASONS | BIKE_COUNT | TEMPTERATURE | HUMIDITY | WIND_SPEED | VISIBILITY | DEW_POINT_TEMP | SOLAR_RADIAT |
|---|---|---|---|---|---|---|---|---|
| | <fct> | <int> | <dbl> | <int> | <dbl> | <int> | <dbl> | <d |
| 1 | Winter | 225 | -2.540463 | 49 | 1.922685 | 1445 | -12.416667 | 0.2981 |
| 2 | Spring | 746 | 13.021389 | 58 | 1.857778 | 1240 | 4.091389 | 0.6803 |
| 3 | Autumn | 924 | 13.821167 | 59 | 1.492101 | 1558 | 5.150594 | 0.5227 |
| 4 | Summer | 1034 | 26.587274 | 64 | 1.609420 | 1501 | 18.750136 | 0.7612 |

```r
# provide your solution here
# total # of bikes available in Seoul....

query <- "SELECT BIKE_SHARING_SYSTEMS.BICYCLES, WORLD_CITIES.CITY,
        WORLD_CITIES.COUNTRY, WORLD_CITIES.LAT,
        WORLD_CITIES.LNG, WORLD_CITIES.POPULATION
        FROM WORLD_CITIES
        INNER JOIN BIKE_SHARING_SYSTEMS
        ON (WORLD_CITIES.CITY_ASCII = BIKE_SHARING_SYSTEMS.CITY) AND
                (WORLD_CITIES.CITY = 'Seoul')
        "
sqlQuery(conn,query)
```

A data.frame: 1 × 6

| | BICYCLES | CITY | COUNTRY | LAT | LNG | POPULATION |
|---|---|---|---|---|---|---|
| | <int> | <fct> | <fct> | <dbl> | <dbl> | <int> |
| 1 | 20000 | Seoul | Korea, South | 37.58 | 127 | 21794000 |